

TP 03 :La généricité et les types génériques

Exercice 1: Exercice 1: Classe Générique Paire

Créez une classe générique appelée Paire qui peut stocker une paire de valeurs de types différents. La classe doit avoir des méthodes pour définir et obtenir chaque valeur de la paire.

Exercice 2: Méthode Générique pour Trouver le Maximum

Écrivez une méthode générique appelée trouverMaximum qui prend un tableau de valeurs comparables et renvoie la valeur maximale dans le tableau.

Exercice 3: Classe Générique Liste Inversée

Créez une classe générique appelée ListeInverse qui peut stocker une liste de valeurs de type générique et renvoie une version inversée de cette liste.

Exercice 4: Méthode Générique pour Fusionner deux Listes

Écrivez une méthode générique appelée fusionnerListes qui prend deux listes de même type et les fusionne en une seule liste, conservant l'ordre des éléments.

Exercice 5: Interface Générique Comparateur

Définissez une interface générique appelée Comparateur avec une méthode comparer qui prend deux objets du même type et renvoie un entier négatif si le premier objet est inférieur au deuxième, zéro s'ils sont égaux, et un entier positif sinon.

Exercice 6:

1. Définissez une classe générique appelée `Noeud` qui représente un nœud dans un arbre binaire de recherche. Chaque nœud doit avoir une valeur de type générique, ainsi que des références vers les nœuds gauche et droit.
2. Implémentez une classe générique appelée `ArbreBinaireRecherche` qui représente l'arbre binaire de recherche. Cette classe doit avoir un champ pour le nœud racine de l'arbre.
3. Ajoutez des méthodes à la classe `ArbreBinaireRecherche` pour insérer une valeur dans l'arbre, rechercher une valeur dans l'arbre et parcourir l'arbre dans l'ordre.
4. Testez votre implémentation en créant plusieurs arbres binaires de recherche avec différents types de données (entiers, chaînes, objets personnalisés, etc.) et en effectuant des opérations d'insertion, de recherche et de parcours.

Voici un exemple de structure de classe pour commencer :

```
```java
class Noeud<T> {
 T valeur;
 Noeud<T> gauche;
 Noeud<T> droit;

 public Noeud(T valeur) {
 this.valeur = valeur;
 }
}
```

```
 gauche = null;
 droit = null;
 }
}

class ArbreBinaireRecherche<T extends Comparable<T>> {
 Noeud<T> racine;

 public ArbreBinaireRecherche() {
 racine = null;
 }

 // Méthodes d'insertion, recherche et parcours de l'arbre
 // À implémenter
}
...

```

#### ### Instructions Supplémentaires:

- Assurez-vous d'implémenter correctement les méthodes d'insertion, de recherche et de parcours de l'arbre binaire de recherche.
- Testez votre implémentation avec différents types de données pour vous assurer qu'elle fonctionne correctement avec des données de différents types.
- Assurez-vous que les valeurs insérées dans l'arbre sont bien ordonnées selon le principe d'un arbre binaire de recherche : les valeurs plus petites sont situées dans le sous-arbre gauche et les valeurs plus grandes dans le sous-arbre droit.

Cet exercice vous permettra de pratiquer l'utilisation des types génériques en Java pour créer une structure de données flexible et réutilisable comme un arbre binaire de recherche. Si vous avez des questions ou avez besoin d'aide pour implémenter les méthodes spécifiques, n'hésitez pas à demander !

#### **Exercice 7:**

Voici un exercice sur la création d'une liste linéaire générique en Java :

#### ### Exercice: Liste Linéaire Générique

1. Définissez une classe générique appelée `Noeud` qui représente un élément de la liste. Chaque nœud doit contenir une valeur de type générique et une référence vers le nœud suivant dans la liste.
2. Implémentez une classe générique appelée `ListeLineaire` qui représente la liste linéaire. Cette classe doit avoir un champ pour le nœud de tête de la liste.
3. Ajoutez des méthodes à la classe `ListeLineaire` pour ajouter un élément en tête de liste, ajouter un élément en queue de liste, rechercher un élément dans la liste et supprimer un élément de la liste.

4. Testez votre implémentation en créant plusieurs listes linéaires avec différents types de données (entiers, chaînes, objets personnalisés, etc.) et en effectuant des opérations d'ajout, de recherche et de suppression d'éléments.

Voici un exemple de structure de classe pour commencer :

```
```java
class Noeud<T> {
    T valeur;
    Noeud<T> suivant;

    public Noeud(T valeur) {
        this.valeur = valeur;
        suivant = null;
    }
}

class ListeLineaire<T> {
    Noeud<T> tete;

    public ListeLineaire() {
        tete = null;
    }

    // Méthodes d'ajout, de recherche et de suppression d'éléments dans la liste
    // À implémenter
}
```
```

### Instructions Supplémentaires:

- Assurez-vous d'implémenter correctement les méthodes d'ajout, de recherche et de suppression d'éléments dans la liste linéaire.
- Testez votre implémentation avec différents types de données pour vous assurer qu'elle fonctionne correctement avec des données de différents types.
- Assurez-vous que les éléments ajoutés dans la liste sont correctement liés les uns aux autres pour former une liste linéaire cohérente.

Cet exercice vous permettra de pratiquer l'utilisation des types génériques en Java pour créer une structure de données flexible et réutilisable comme une liste linéaire. Si vous avez des questions ou avez besoin d'aide pour implémenter les méthodes spécifiques, n'hésitez pas à demander !